Please expand the documentation in the following code.

```
def merge_dicts(base, override):
    """
    Merge two dictionaries, with values from override taking precedence.
    Returns a new dictionary; neither input is mutated.
    """
    result = base.copy()
    for key, value in override.items():
        if isinstance(value, dict) and key in result and isinstance(result[key], dict):
            result[key] = merge_dicts(result[key], value)
        else:
            result[key] = value
    return result
```

Here's an expanded docstring that explains the behavior, parameters, return value, recursion rules, and provides examples:

Python Run

```python
def merge_dicts(base, override):
    """
    Recursively merge two dictionaries and return the merged result.

    This function creates a new dictionary by combining the contents of
    ``base`` and ``override``. Values from ``override`` take precedence over
    values from ``base`` when the same key exists in both dictionaries.

    If a key exists in both dictionaries and the corresponding values are
    themselves dictionaries, those nested dictionaries are merged
    recursively. For all other value types, including lists, tuples, sets,
    and scalar values, the value from ``override`` completely replaces the
    value from ``base``.

    Neither input dictionary is modified. A new dictionary object is returned.

    Parameters
    ----------
```

```
base : dict
    The default or original dictionary whose values provide the starting
    state for the merge.

override : dict
    The dictionary containing values that should override or extend the
    contents of ``base``.

Returns
-------
dict
    A new dictionary containing the merged contents of ``base`` and
    ``override``.

Notes
-----
- Nested dictionaries are merged recursively.
- Non-dictionary values are replaced rather than merged.
- The function performs a shallow copy of dictionaries at each level
  before applying recursive merges.
- Objects stored as values are not deep-copied unless they are merged
  through recursive dictionary processing.

Examples
--------
Basic override:

>>> merge_dicts(
...     {"host": "localhost", "port": 8080},
...     {"port": 9090}
... )
{'host': 'localhost', 'port': 9090}

Recursive merge:

>>> merge_dicts(
...     {"database": {"host": "db1", "port": 5432}},
...     {"database": {"port": 5433}}
... )
{'database': {'host': 'db1', 'port': 5433}}

Adding new keys:

>>> merge_dicts(
...     {"debug": False},
...     {"timeout": 30}
... )
{'debug': False, 'timeout': 30}

Replacing non-dictionary values:

>>> merge_dicts(
```

```
...     {"items": [1, 2, 3]},
...     {"items": [4]}
... )
{'items': [4]}
"""
result = base.copy()
for key, value in override.items():
    if isinstance(value, dict) and key in result and isinstance(result[key], dict):
        result[key] = merge_dicts(result[key], value)
    else:
        result[key] = value
return result
```

This style is suitable for internal libraries and API documentation generators such as Sphinx, NumPy-style doc tooling, and IDE documentation popups.

Maintaining the documentation style established in the previous conversation, please document the new functions in the following code.

```
def load_config(path):
    # open file
    with open(path) as f:
        # parse and return
        return json.load(f)


def save_config(path, data):
    # write json
    with open(path, "w") as f:
        json.dump(data, f, indent=2)
```

To match the NumPy-style documentation used for `merge_dicts`, you could document the new functions as follows:

Python Run

```
def load_config(path):
    """
    Load configuration data from a JSON file.
```

```
    This function opens the JSON file located at ``path``, parses its
    contents using ``json.load()``, and returns the resulting Python object.
    The returned value is typically a dictionary, but may be any JSON-compatible
    type depending on the file contents.

    Parameters
    ----------
    path : str or pathlib.Path
        Path to the JSON configuration file to read.

    Returns
    -------
    object
        The Python object produced by deserializing the JSON content.
        This is commonly a dictionary representing configuration settings.

    Raises
    ------
    FileNotFoundError
        If the specified file does not exist.

    json.JSONDecodeError
        If the file contains invalid JSON.

    OSError
        If an operating system error occurs while opening or reading
        the file.

    Examples
    --------
    Load a configuration file:

    >>> config = load_config("config.json")
    >>> config["host"]
    'localhost'

    Load nested configuration data:

    >>> config = load_config("settings.json")
    >>> config["database"]["port"]
    5432
    """
    with open(path) as f:
        return json.load(f)


def save_config(path, data):
    """
    Save configuration data to a JSON file.

    This function serializes ``data`` as JSON and writes it to the file
    specified by ``path``. Existing file contents are overwritten. The JSON
```

```
output is formatted with an indentation level of two spaces to improve
readability.

Parameters
----------
path : str or pathlib.Path
    Path to the file where the JSON data should be written.

data : object
    The JSON-serializable Python object to write. This is typically a
    dictionary containing configuration settings, but may be any object
    supported by ``json.dump()``.

Returns
-------
None

Raises
------
TypeError
    If ``data`` contains values that cannot be serialized to JSON.

OSError
    If an operating system error occurs while opening or writing
    the file.

Notes
-----
- Existing file contents are replaced.
- Output is formatted using ``indent=2`` for human readability.
- The function does not create missing parent directories.

Examples
--------
Save a configuration dictionary:

>>> save_config(
...     "config.json",
...     {"host": "localhost", "port": 8080}
... )

Save nested configuration data:

>>> save_config(
...     "settings.json",
...     {
...         "database": {
...             "host": "db1",
...             "port": 5432
...         }
...     }
... )
```

```
    """
    with open(path, "w") as f:
        json.dump(data, f, indent=2)
```

This keeps the same structure, level of detail, and section ordering as the expanded `merge_dicts` documentation.